Phase 7 | Security, Correctness & Failure Analysis

Break the assumptions safely and improve validation.

FastAPI Edition | Repository-grounded study material | Snapshot: 2026-10-24

How to read this phase

A risk is not automatically a verified vulnerability. The table below separates current evidence, scenario, impact, test, and mitigation. Do not run destructive tests against a deployed service or real data. Reproduce with local test fixtures.

1. Unknown station accepted as fallback (correctness risk)

Evidence: `get_station_info` returns Unknown metadata; `predict_hourly_footfall` defaults unknown station index to 0; routes don't first check station membership.

Precondition: request uses syntactically valid station label absent from CSV.

Impact: response may look valid but use an unrelated station's index/statistics.

Safe test: unit test helper with an unknown station and assert current behavior; no external service needed.

Mitigation: canonical station lookup before inference; return 404/422 with allowed station search endpoint; never silently map unknown to index zero.

2. Missing/corrupt artifacts prevent startup (availability risk)

Evidence: `joblib.load` executes at module import; station and operations CSVs are also read on import.

Precondition: artifact missing, corrupt, incompatible, or unexpected working directory.

Impact: import/server startup fails.

Safe test: isolated temporary copy/test monkeypatch paths; don't rename production assets.

Mitigation: explicit artifact manifest/checksums, clear startup validation, controlled lifespan loader, smoke test, deployment packaging audit. Keep trusted artifacts only because pickle-like joblib may execute code during loading.

3. Unmatched topology label silently drops edges (route correctness)

Evidence: `mapping.get()` returns None and adjacent pair is skipped; no startup failure is raised.

Precondition: route label not normalized/aliased to station table.

Impact: reachable destination count decreases or path detours unexpectedly.

Safe test: enumerate topology names versus canonical mappings in a read-only audit; build a tiny fixture containing one mismatch.

Mitigation: report all unmatched names, alias review, fail CI on missing expected interchanges, use stable station IDs rather than display labels.

4. Normalization collision (data integrity)

Evidence: normalized station dictionary keys may overwrite if two names normalize to same key.

Impact: route attaches to wrong physical station without obvious error.

Test: compute normalized key counts and assert no collision.

Mitigation: map normalized names to lists; require explicit disambiguation; use stable IDs.

5. Synthetic evaluation mistaken for real validation (model misuse)

Evidence: evaluation JSON labels data as synthetic; API returns `SYNTHETIC_SIMULATION_ONLY` and real-world validation false.

Impact: stakeholders could treat high scores as operationally reliable.

Test: verify dashboard/API communicates data basis and generated-vs-observed status.

Mitigation: persistent UI caveat, provenance in exports, real held-out count collection, calibration, independent review.

6. Historical feature leakage or drift

Evidence: pipeline carefully uses prior dates for training features and training period aggregates for test; future inference uses those saved aggregates.

Scenario: later feature engineering accidentally uses same-day/future targets or a future holiday label unavailable at forecast date.

Impact: optimistic metrics or unavailable production feature.

Test: assert each feature's source timestamp <= forecast timestamp; construct boundary-date fixture; compare against simple baseline.

Mitigation: point-in-time feature tests, chronological cross-validation, versioned feature contract, leakage review.

7. Overload and resource exhaustion

Scenario: repeated large profile requests or high concurrency consumes threadpool/CPU; output includes all destinations.

Evidence: synchronous prediction and DataFrame construction are delegated to a bounded threadpool; no explicit rate limit shown.

Impact: latency growth and worker saturation.

Safe test: local load test with small controlled request count and test server only.

Mitigation: input bounds, caching/vectorized prediction, concurrency limits, monitoring, rate limits at edge; queue only if justified.

8. Malformed/oversized request data

Evidence: malformed JSON and non-object JSON are handled; no typed Pydantic request model or request-size policy is apparent.

Impact: unhelpful validation errors or excessive memory/read work.

Test: local unit tests for malformed JSON, wrong shapes, invalid date/time, huge body policy.

Mitigation: Pydantic schema, max request body size, enum/range validation, consistent 400/422 responses.

9. Dependency and artifact supply chain

Risk: broad version ranges and joblib deserialization compatibility; loading untrusted artifacts is dangerous.

Impact: breaking deployment or code execution during load.

Mitigation: lockfile, trusted build provenance, artifact hashes/signatures, dependency scanning and controlled upgrades. Do not upload secrets or private ridership data to repository.

10. Sensitive data and API controls

Current prototype has no authentication and uses synthetic data, so user identity/privacy risk is limited today. If real passenger movement data or admin features are added, classify data, aggregate/minimize, set retention, access control, audit, TLS, secret management, and disclosure policy. Do not claim authentication exists when it does not.

11. Operational assumption correctness

Capacities/headways/transfer penalties/through-load factors are configurable assumptions. A wrong value can generate a confident but wrong recommendation. Maintain provenance and effective dates; separate observed facts from assumptions; surface units; use sensitivity ranges and independent domain review.

12. Failure recovery checklist

On error: capture request ID and sanitized stack in logs; return a stable error schema; do not expose local filesystem paths; keep service health distinct from model readiness; deploy known-good version and rollback; verify model/data manifest. For prediction faults, never silently substitute zeros or station 0 without a visible status.

How to use this series

Read Phase 1 first, then Phase 2 while opening the source files. Use Phases 3, 5, 6, and 7 to deepen technical understanding; rehearse with Phases 4 and 8; revise with Phase 9. The editable Markdown beside each PDF is the source of truth for corrections.

Evidence and honesty rules

MetroInsights is a planning prototype. Its demand targets and holdout are synthetic. Model metrics describe agreement with generated scenario labels, not real passenger counts or operational DMRC accuracy. Network topology, coordinates, capacities, headways, service frequency, and interchange splits are project data or configurable assumptions unless separately verified. Say “estimated” and explain the assumptions. Do not describe this as a live passenger-count, train occupancy, or production DMRC system.

Verified snapshot used in this edition

The cleaned station table contains 243 station records. The topology graph's destination count depends on the selected source and excludes the source itself; 242 destinations can be returned when all other modeled stations are connected. Do not say 226 or 228 without checking the current data. The project defines 11 route keys, including branch keys. The synthetic training series uses 243 stations x 728 dates x 19 hourly slots (05:00-23:00), or 3,361,176 rows. A chronological holdout begins 2026-10-01 and has 438,615 rows. The recorded HistGradientBoostingRegressor scores are MAE 26.58, RMSE 51.10, R2 0.9946, and MAPE 5.18%; Ridge MAE is 53.67 and historical station/day/hour mean MAE is 60.25. These are synthetic-only evaluation results.

Current stack and constraints

FastAPI + Uvicorn, Starlette/Jinja2, server-rendered HTML with JavaScript/CSS, Pandas, NumPy, scikit-learn, joblib, CSV/JSON, and a custom adjacency graph with heapq Dijkstra. No relational database, live ridership feed, authentication, or distributed job queue is present. The generated multi-million-row CSV is a training input; runtime inference loads compact serialized model/metadata plus station and operations inputs, rather than reading every training row on each request. Runtime startup imports routes and eagerly loads artifacts/builds the graph, so missing files can prevent startup. The current FastAPI migration has passed static compilation and diff checks in prior work, but an actual runtime smoke test was not possible in the offline environment; verify locally before claiming deployment verification.